Edutor AI

EdutorAI DOCUMENTATION

1. Introduction

- Project Title : Health AI: Intelligent Healthcare Assistant using IBM granite
- Team member: Rakesh A
- Team member: Sanjay p
- Team member: Sarath Kumar k
- Team member: snekan v

2. Project Overview:

EdutorAI is an intelligent healthcare assistant that leverages IBM Granite large language models to provide safe and useful support for patients and clinicians. It can answer medical queries, assist with symptom triage, summarize health records, and suggest possible next steps ,

such as booking an appointment or contacting emergency services.
The system is designed as a conversational AI platform with secure integration of medical knowledge bases and strict handling of sensitive health data.

3. Architecture:

High-level components:

Frontend (React): chat UI, clinician dashboard, conversation history, attachments (images).

API Gateway / Backend (FastAPI or Express): authentication, rate-limits, routing, logging, orchestration of model calls.

Model Runtime:

Option A — Local: run Granite via Ollama or local model runner (Hugging Face weights).

Good for dev / offline. Option B — Cloud: host models via IBM watsonx for scalable, enterprise deployments.

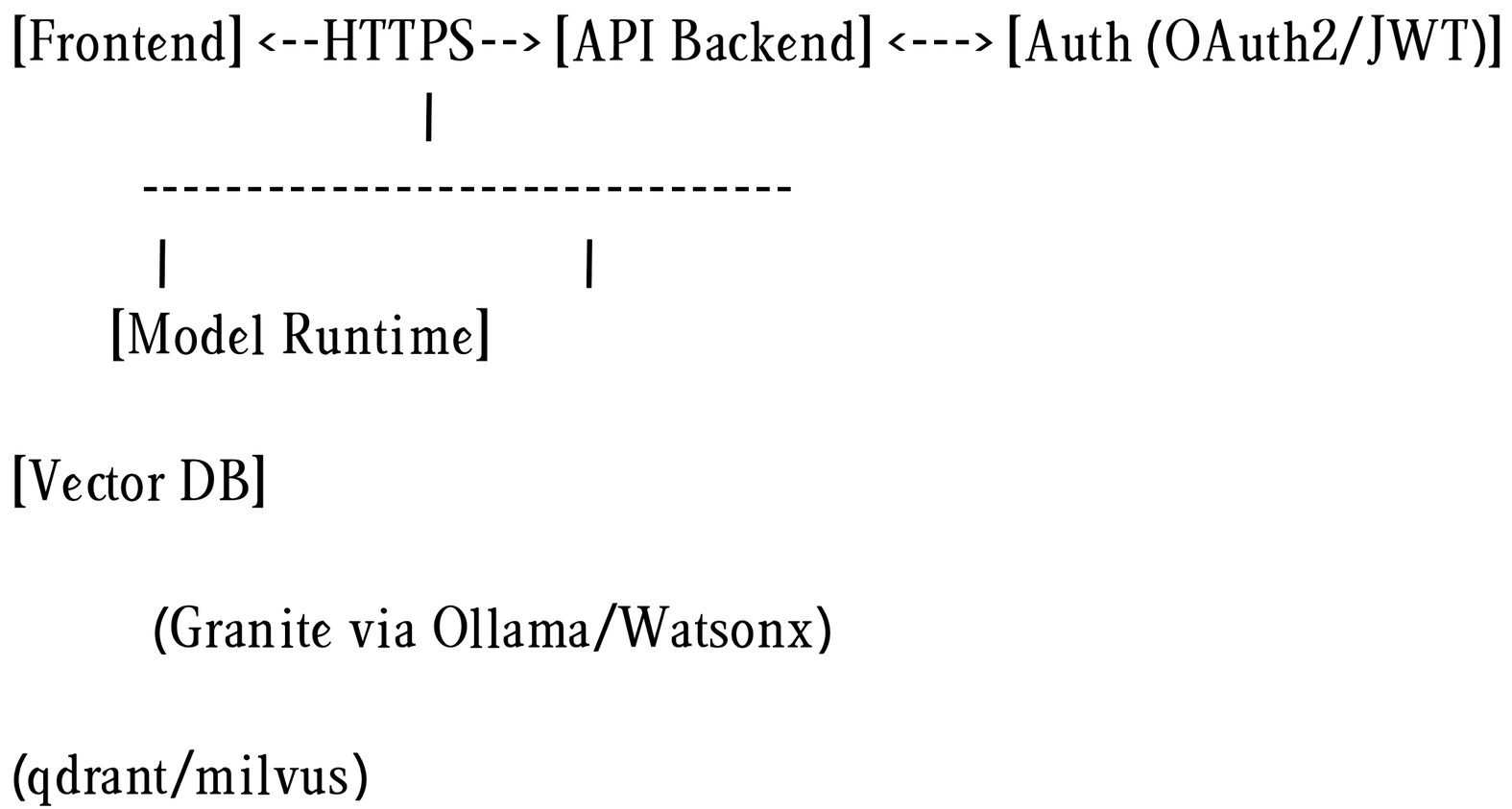
Vector DB (Milvus / Pinecone / Weaviate / qdrant): for RAG retrieval of medical docs and guidelines.

Database (Postgres / MySQL): user profiles, conversation metadata (PHI encrypted).

Storage (S3-compatible): screenshots, reports, attachments.

Monitoring: Prometheus + Grafana, and audit logs for security.

Diagram (ASCII sketch):



4. Setup Instructions

To set up the system:

1.Clone the project repository from version control.

2. Copy the .env.example file to .env and fill in configuration values such as database URL, API keys, and Granite endpoints.

3. Install dependencies for both backend and frontend.

4.For local development, use Docker Compose to start services like backend,frontend, database, and vector database.

5. For production, deploy containers on Kubernetes and configure IBM watsonx for Granite model hosting.

5. Folder Structure

The project is organized to keep backend, frontend, and infrastructure code separate:

```
health-ai/
├── backend/      # FastAPI backend, APIs, services
├── frontend/     # React-based user interface
├── infra/        # Deployment scripts and Docker/Kubernetes files
├── docs/         # Documentation and diagrams
├── tests/        # Unit, integration, and e2e tests
└── README.md     # Main project overview
```

This modular structure supports scalability and team collaboration.

6. Running the Application

Development Mode:

Start the backend with Uvicorn (uvicorn app.main:app --reload)

Start the frontend with npm run dev.

Docker Mode:

Use docker-compose up to runbackend, frontend, database, and vector DB in containers.

Production Mode:

Build Docker images, push them to a registry, and deploy them using Kubernetes with TLS and autoscaling enabled.

7. API Documentation:

The backend provides REST APIs for communication:

POST /chat → Sends user messages to Granite models and returns assistant responses

GET /conversations/{id} → Retrieves conversation history.POST /auth/login → Authenticates users and issues JWT tokens.

POST /admin/upload-docs → Allows admins to upload documents into the vector database for RAG.

All APIs use secure authentication and model calls are routed only through the backend for safety.

8. Authentication

The application uses OAuth2 with JWT tokens to ensure secure login and API access.

Sensitive information like API keys and patient records are encrypted before storage.

Short-lived tokens reduce security risks.

PHI (Personal Health Information) is stored only in compliance with HIPAA and GDPR standards.

9. User Interface

The system has two main interfaces:

Patient Chat UI:

A conversational interface for patients to ask questions, report symptoms, and receive triage guidance. Attachments like medical images can also be uploaded.

Clinician Dashboard:

A panel for healthcare providers to review patient conversations, summaries, and triage outcomes. It includes evidence sources and allows integration with electronic health records.

The UI follows WCAG 2.1 AA accessibility standards, ensuring it is usable for people with disabilities.

10. TestingTesting

Ensures accuracy, reliability, and safety:

Unit Testing: Covers API endpoints, authentication, and vector DB queries.

Integration Testing: Validates the RAG pipeline with Granite models.

End-to-End Testing: Simulates complete workflows from patient query to clinician review.

Safety & Evaluation Testing: Benchmarks the system against medical QA datasets and detects hallucinations or unsafe responses.

11. Screenshots

Screenshots are provided for clarity and demonstration:

Login and signup screens.

Patient chat interface with Granite responses.

Clinician dashboard showing conversation summaries.

Admin upload panel for knowledge base management.

All screenshots should exclude any personal health data and only display sample/demo content.

12. Known Issues

Hallucinations: Granite may sometimes generate incorrect information without strong RAG grounding.

Latency: Large models increase response time, especially on low-end hardware.

Cost: Running Granite in the cloud may be expensive without optimization.

Regulatory Constraints: Full compliance with healthcare regulations requires ongoing monitoring.

13. Future Enhancements

Planned improvements include:

Multimodal Input: Support for processing images like skin rashes or X-rays using Granite vision models.

Clinical Fine-Tuning: Training Granite on domain-specific datasets for higher accuracy.

Explainability Features: Displaying evidence, confidence scores, and reasoning steps.

Offline Support: Running smaller Granite models locally for clinics with limited internet access.

Clinician Approval Workflows: Adding human-in-the-loop validation before critical advice is sent to patients.